15 10 2026

1.05.01 Object-Oriented Modeling Practices

Object-Oriented (OO) modeling relies on foundational design best practices from the very beginning of software development. A core rule is to keep class variables strictly private to uphold the encapsulation principle. Conceptual modeling is used to capture ideas, understand real-world systems, and communicate effectively across all stages of development. To support this, UML provides essential diagram types such as use case, class, interaction, and state charts

Architecturaly, software should be organized into loosely coupled layers (User Interface, Application Logic, Persistence, Communication and Operating System) where each layer build on lower-level abstractions. When deciding on code reuse, prefer aggregation over inheritance because aggregation offers greater flexibility since relationships can be modified at runtime. Additionally, programming to an interface or abstraction helps minimize coupling and removes unnecessary dependencies

Finally, readability, maintainability, and code quality are driven by naming conventions and testing. Classes should use nouns in PascalCasing variables should use lowerlamelCasing, and methods should use action verbs

1.05.07 Introduction to UML

The Unified Modeling Languag (UML) is an expressive, semi-formal and process-independent collection of object-ouented modeling languages that support increment development. UML consists of building blocks where modeling elements act like words, diagrams like sentences, and models like entire books. Systems are analyzed trought different views, wich corespod to three main software engineering perspectives

- Analysis Perspective
- Specification Perspective
- Implementation Perspective

UML Diagrams

1) Structural Diagrams show the static elements of a system, their physical or conceptual components, and how they relate to one another. They represent the "skeleton" or architecture of the software

2) Behavioral Diagrams show what happens inside the system while is it running. They capture the dynamic aspects, time dependent interactions, and how data or control flows between objects over time

1.05.08 Basic Use Cases

What is an Actor? Actors represent anything external to the system that needs to interact with it, such as users, other software systems, or hardware devices. They are classified into sets denoted by stick figures with singular, descriptive names. It can also have generalization/specialization relationships

What is a Use Case? Capture and document the user-visible functionality of a system, representing the goals that users need to accomplish independently of specific features. They answer "What" and "Why" questions

Use-Case Relationships

- <<extends>> Models variations or optional enhancements
- <<include>> Defines how a use case requires the functionality
- Generalization Indicates that one use case is a specialized subset of a more general use case

1.05.09 Class Diagrams

- **Objects:** Real-world entities with identity, state, and behavior
- **Classes:** Abstractions grouping similar objects named with singular nouns in Upper Camel Case
- **Three Perspectives**
  - Analysis
  - Specification
  - Implementation
- **Class Structure** Represented as a box divided into Name, Attributes and Operations
- **Generalization** Inheritance / specialization
- **Association** Structural connections between classes with multiplicity
- **Dependency** A "uses" relationship represented by a dashed arrow

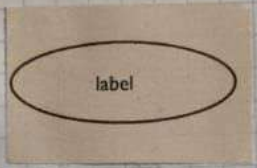
1.05.10 UML References

Use Case Diagrams

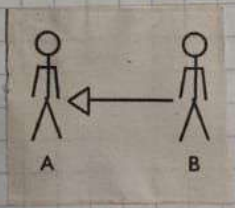
Actor Someone or something that interacts with it achieves a goal



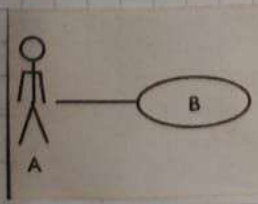
Use Case A discrete goal that an actor needs to accomplish with respect to the system free of design choices



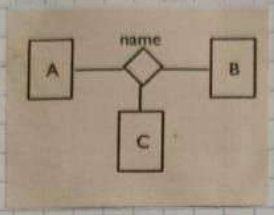
Generalization / Specialization The set of B is a subset of the set of A. Therefore every on B is also in A



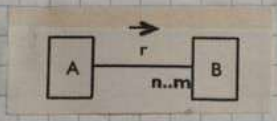
Actor "has" Use Case Actor "A has Use Case B" respect to the system



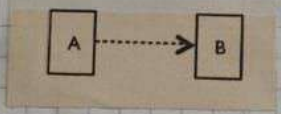
Named N-ary Association Connects three or more classes brought a central diamond shape



Multiplicity Constraint An A object can be related to between n and m B objects



Dependency An A object may depend on a B object



Attributes Properties or placeholders for values that objects in a class possess

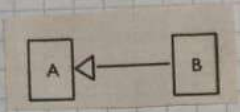
Notes

Class Diagram

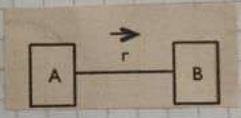
Class A set of objects related to the application with common characteristics



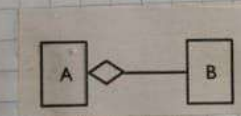
Generalization / Specialization "B is a A" (Set B is subset of set A)



Named Binary Association A set of persistent links between objects (A and B) described by a clear verb



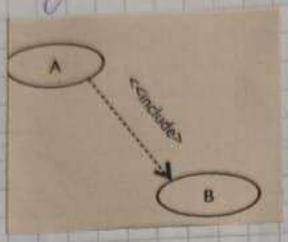
Aggregation "A contains B"



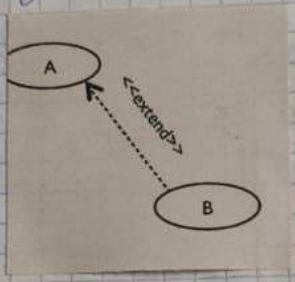
Composition If the object A is destroyed all its subparts B are also destroyed



Include Dependency The goal A includes the sub-goal B



Extend Dependency The goal B adds to or alters goal A



Note Any note can be connected to particular component with a dotted line

